RoboRAG: A Retrieval-Augmented Generation Web Application for Robot Task Planning and Motion Planning Assistance

DS552 — Generative AI

Worcester Polytechnic Institute

Submitted by:

Piyush Thapar

Instructor:

Dr. Narahara Chari Dingari

April 2026

Contents

1 Project Title **3**

2 Introduction and Objective **3**
2.1 Introduction . . . 3
2.2 Objective . . . 3
2.3 Why RAG with GPT-4o-mini Is Suitable . . . 3

3 Selection of Generative AI Model **4**
3.1 Model Choice: OpenAI GPT-4o-mini . . . 4
3.2 Embedding Model: OpenAI text-embedding-3-small . . . 4
3.3 Alternatives Considered . . . 4

4 Project Definition and Use Case **5**
4.1 Application Concept . . . 5
4.2 How the Generative AI Model Is Integrated . . . 5
4.3 Use Case Scenario . . . 5
4.4 System Architecture . . . 5

5 Implementation Plan **5**
5.1 Technology Stack . . . 5
5.2 Web Framework: Streamlit . . . 5
5.3 Development Steps . . . 6

6 Model Evaluation and Performance Metrics **7**
6.1 Evaluation Methodology . . . 7
6.2 Generation Quality — RAG vs. Baseline . . . 7
6.3 Retrieval Quality . . . 7
6.4 Efficiency . . . 8
6.5 Per-Question Breakdown . . . 8
6.6 User Experience . . . 9
6.6.1 Multi-Turn Conversation Memory . . . 9
6.6.2 Retrieval Confidence Indicator . . . 9
6.6.3 AI-Generated Follow-Up Suggestions . . . 10
6.6.4 Answer Feedback System . . . 10

7 Deployment Strategy **10**
7.1 Deployment Platform: Streamlit Cloud . . . 10
7.2 Deployment Process . . . 10
7.3 Why Streamlit Cloud . . . 10
7.4 Expected User Interaction . . . 10

8 Expected Outcomes and Challenges **11**
8.1 Expected Outcomes . . . 11
8.2 Challenges Encountered and Solutions . . . 11

9 Resources Required **12**
9.1 Software and Frameworks . . . 12

9.2 Models 12
9.3 Data Sources (Curated Knowledge Base) 12
9.4 Hardware and Cost 12

10 Conclusion **13**
10.1 Key Takeaways 13
10.2 Potential Improvements and Future Enhancements 13

1. Project Title

RoboRAG: A Retrieval-Augmented Generation Web Application for Robot Task Planning and Motion Planning Assistance

Deployed Application: https://roborag.streamlit.app/

GitHub Repository: https://github.com/ThaparPiyush/RoboRAG

2. Introduction and Objective

2.1 Introduction

Robotics researchers and engineers face a recurring challenge: the knowledge they need to make informed decisions about motion planners, manipulation strategies, sensor configurations, and software frameworks is scattered across academic papers, documentation pages, tutorials, and forum discussions. A graduate student trying to choose between RRT* and STOMP for a 7-DOF arm might need to search through three arXiv papers, a half-finished ROS Answers thread, and a MoveIt GitHub issue before finding a useful answer.

Large Language Models (LLMs) have demonstrated impressive natural-language understanding and generation capabilities, but they frequently hallucinate facts in specialized technical domains. In robotics specifically, LLMs have been observed to confidently recommend API functions that do not exist or describe algorithm behaviors that are factually incorrect. This limits their usefulness as reliable technical assistants.

Retrieval-Augmented Generation (RAG) [1] addresses this fundamental limitation by grounding the LLM's responses in actual retrieved documents. Rather than relying on the model's parametric memory, RAG retrieves the most relevant passages from a curated knowledge base at inference time and instructs the LLM to synthesize its answer based on that context. This significantly improves factual accuracy and provides verifiable source citations.

2.2 Objective

The objective of this project is to develop **RoboRAG**, a web application that serves as an intelligent question-answering assistant for robot task planning and motion planning. Users can ask natural-language questions such as:

- "What motion planner should I use for a 7-DOF arm in a cluttered environment?"
- "How does RRT* differ from PRM for high-dimensional configuration spaces?"
- "What are the best practices for grasping transparent objects?"

The application retrieves relevant passages from a curated knowledge base of robotics documentation, then uses OpenAI's GPT-4o-mini model to synthesize a grounded, coherent answer along with source citations.

2.3 Why RAG with GPT-4o-mini Is Suitable

A RAG architecture is ideal for domain-specific technical Q&A because:

1. **Factual grounding:** The LLM reads relevant documentation at inference time rather than relying on potentially outdated or incorrect parametric memory.
2. **Source transparency:** Every answer comes with retrievable source passages that the user can verify, which is critical in engineering domains.
3. **Focused domain coverage:** By curating the knowledge base, we control exactly what the system knows and can ensure comprehensive coverage without noise.
4. **High-quality generation:** GPT-4o-mini provides excellent instruction-following and summarization at very low cost (~$0.15 per 1M input tokens).

3. Selection of Generative AI Model

3.1 Model Choice: OpenAI GPT-4o-mini

The project uses **OpenAI GPT-4o-mini** as the generative model, accessed through the OpenAI API via the `langchain-openai` integration.

Criterion	Justification
Performance	Excels at reading comprehension, summarization, and instruction-following
Cost	$0.15/1M input tokens, $0.60/1M output tokens — under $2 total
API reliability	Production-grade, generous rate limits, no rate-limit issues
LangChain support	First-class integration via `langchain-openai`
Context window	128K tokens — easily accommodates retrieved chunks + question
Latency	Average 3–5 seconds per response

Table 1: Model selection criteria and justification.

3.2 Embedding Model: OpenAI text-embedding-3-small

For vector embeddings, the project uses OpenAI's `text-embedding-3-small` model, which converts both document chunks and user queries into 1536-dimensional dense vectors for semantic similarity search. At $0.02 per 1M tokens, embedding the entire knowledge base costs less than $0.01. Using OpenAI's embedding model eliminates the need for a local ML model, reducing deployment memory requirements significantly.

3.3 Alternatives Considered

- **Google Gemini 2.5 Flash (free tier):** Initially tested but the free tier had severe rate limits (20 generation requests/day) that made development and reliable deployment impractical.
- **Mistral 7B (local via Ollama):** Requires 4.5 GB RAM and cannot run on free cloud hosting platforms.
- **Groq API (free Mistral):** Free but has reliability issues under load.

GPT-4o-mini provides the best combination of quality, reliability, generous rate limits, and extremely low cost.

4. Project Definition and Use Case

4.1 Application Concept

RoboRAG is a **domain-specific question-answering system** for robotics and motion planning, built as an interactive web application with a chat interface. It is specifically designed to answer technical questions about motion planning algorithms, the MoveIt 2 framework, ROS 2, robot manipulation, kinematics, and related topics.

4.2 How the Generative AI Model Is Integrated

The application follows a Retrieval-Augmented Generation (RAG) architecture with two stages:

Offline stage (one-time ingestion):

1. Curated robotics documents are loaded and parsed (8 documents, ~15,000 words).
2. Documents are split into overlapping chunks (~800 tokens, 100-token overlap) using LangChain's `RecursiveCharacterTextSplitter`.
3. Each chunk is embedded using OpenAI's `text-embedding-3-small`.
4. Vectors are stored in ChromaDB with metadata (source title, file path).

Online stage (per user query):

1. The user types a question in the Streamlit chat interface.
2. The question is embedded using the same embedding model.
3. ChromaDB retrieves the top-5 most similar document chunks.
4. Retrieved chunks + question + system prompt are assembled into a prompt.
5. GPT-4o-mini generates a grounded answer with source citations.
6. The answer and source passages are displayed in the chat interface.

4.3 Use Case Scenario

A robotics graduate student setting up a manipulation pipeline asks: "What motion planner should I use for a 7-DOF arm in a cluttered environment?" Within seconds, they receive an answer recommending RRT-Connect as a fast feasible path finder, STOMP for smooth trajectories, and Pilz for deterministic motion—all with citations to specific knowledge base documents. The student can expand the "Source Documents" section to verify claims.

4.4 System Architecture

Figure 1 illustrates the end-to-end RAG pipeline, from user query to grounded answer.

5. Implementation Plan

5.1 Technology Stack

5.2 Web Framework: Streamlit

Streamlit was chosen because it is native Python (no frontend code required), provides built-in `st.chat_input` and `st.chat_message` components for a polished chat interface,

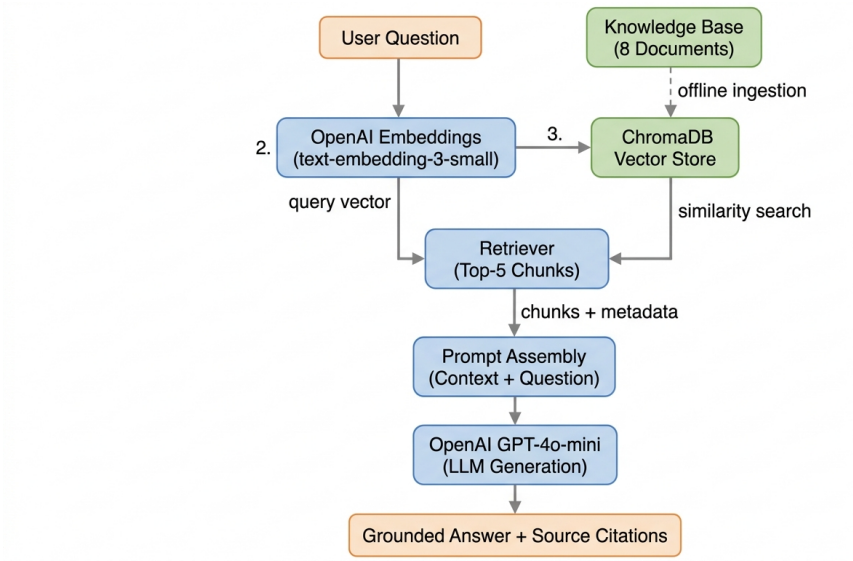

Figure 1: RoboRAG system architecture: Retrieval-Augmented Generation pipeline.

Layer	Technology	Purpose
LLM	OpenAI GPT-4o-mini	Answer generation
Embeddings	OpenAI text-embedding-3-small	Document and query embedding
Vector Database	ChromaDB (local, persisted)	Semantic similarity search
RAG Orchestration	LangChain	Chain construction, retrieval, prompting
Web Framework	Streamlit	Interactive chat UI
Language	Python 3.10+	All components
Document Processing	LangChain text splitters	Chunking
Evaluation	rouge-score, scikit-learn, psutil	Performance metrics
Configuration	python-dotenv	API key management

Table 2: Complete technology stack.

supports `st.cache_resource` for caching the RAG pipeline across reruns, and Streamlit Cloud offers free, persistent hosting with one-click deployment from GitHub.

5.3 Development Steps

1. **Data curation:** Authored 8 comprehensive markdown documents covering motion planning algorithms, MoveIt 2, ROS 2, robot manipulation, kinematics, path planning, SLAM, and common robot platforms (~15,000 words total).
2. **Ingestion pipeline** (`ingest.py`): Reads knowledge base files, chunks them (800 tokens, 100 overlap), embeds with `text-embedding-3-small`, and stores in ChromaDB. Completes in under 15 seconds.
3. **RAG chain** (`rag_chain.py`): LangChain chain that retrieves top-5 chunks with relevance scores, formats them with source labels, builds a prompt with conversation history and strict system instructions, and generates via GPT-4o-mini.
4. **Streamlit application** (`app.py`): Chat interface with the following features:

- Randomized example questions for onboarding
- Multi-turn conversation memory (last 3 exchanges included in prompt)
- Retrieval confidence indicator (High/Medium/Low based on similarity scores)
- AI-generated follow-up question suggestions (validated against the knowledge base)
- Thumbs up/down feedback buttons on every answer
- Expandable source document panels and response time display

5. **Evaluation** (`evaluate.py`): Automated evaluation against 20 benchmark Q&A pairs measuring ROUGE, F1, retrieval precision, latency, and memory. Includes no-RAG baseline comparison.

6. **Deployment:** Pre-built ChromaDB committed to GitHub; Streamlit Cloud loads it at startup and serves the app at a persistent URL.

6. Model Evaluation and Performance Metrics

6.1 Evaluation Methodology

The system was evaluated against 20 hand-crafted Q&A pairs covering the full breadth of the knowledge base. Each pair includes a question, a reference answer, and relevant keywords for retrieval precision assessment. A **no-RAG baseline** (GPT-4o-mini answering the same questions without retrieved context) was run for direct comparison.

6.2 Generation Quality — RAG vs. Baseline

Metric	RAG (RoboRAG)	Baseline (No RAG)	Improvement
ROUGE-1	0.4943	0.2009	+0.2934 (+146%)
ROUGE-2	0.2541	0.0637	+0.1904 (+299%)
ROUGE-L	0.3666	0.1289	+0.2377 (+184%)
Token F1	0.2635	0.1039	+0.1596 (+154%)

Table 3: Generation quality: RAG vs. standalone LLM baseline (20 questions).

RAG delivers a **146% improvement in ROUGE-1** and a **299% improvement in ROUGE-2** over the standalone LLM. This confirms that retrieval grounding is essential for domain-specific Q&A—the LLM alone lacks sufficiently detailed knowledge of robotics-specific topics.

Figure 2 visualizes the comparison across all four metrics.

6.3 Retrieval Quality

Metric	Score	Interpretation
Precision@5	0.7700	77% of retrieved chunks contain relevant keywords

Table 4: Retrieval quality metrics.

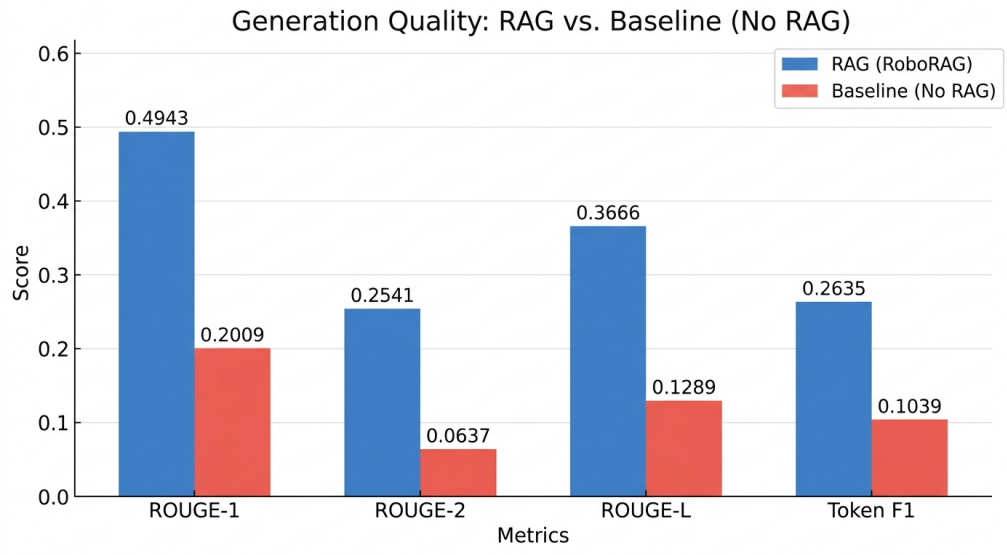

Figure 2: Generation quality comparison: RAG vs. standalone LLM baseline across 20 benchmark questions.

A Precision@5 of 0.77 means that on average, 3.85 out of 5 retrieved chunks are relevant to the question, demonstrating that the embedding model and chunking strategy effectively identify relevant passages.

6.4 Efficiency

Metric	RAG (RoboRAG)	Baseline (No RAG)
Average latency	4.89 s	8.39 s
Peak memory	340.5 MB	—

Table 5: Efficiency metrics.

The RAG pipeline is faster than the baseline because GPT-4o-mini generates shorter, more focused answers when given relevant context. Memory consumption of 341 MB is well within Streamlit Cloud's 1 GB limit.

6.5 Per-Question Breakdown

#	Question (abbreviated)	R-1	R-L	F1	P@5	Latency
1	Motion planner for 7-DOF arm	0.385	0.213	0.212	0.60	4.2 s
2	RRT* vs PRM	0.401	0.234	0.174	1.00	5.5 s
3	Grasping transparent objects	0.515	0.371	0.289	0.60	3.9 s
4	MoveIt 2 planning pipeline	0.354	0.221	0.172	1.00	4.5 s
5	IK solvers in MoveIt 2	0.565	0.488	0.352	1.00	3.8 s
6	STOMP vs CHOMP	0.468	0.194	0.255	1.00	6.1 s
7	ROS 2 communication	0.363	0.187	0.191	1.00	3.7 s
8	Jacobian matrix in robotics	0.356	0.300	0.152	1.00	8.5 s
9	Pick-and-place pipeline	0.518	0.464	0.180	1.00	3.2 s

#	Question (abbreviated)	R-1	R-L	F1	P@5	Latency
10	tf2 in ROS 2	0.516	0.393	0.317	0.80	4.4 s
11	Advantages of redundant arms	0.613	0.438	0.397	0.40	2.0 s
12	Collision checking in MoveIt 2	0.613	0.540	0.337	1.00	3.7 s
13	Forward vs inverse kinematics	0.450	0.344	0.236	0.60	5.1 s
14	SLAM approaches	0.407	0.348	0.234	0.60	4.9 s
15	MoveIt Servo real-time control	0.601	0.417	0.360	0.60	2.8 s
16	Narrow passage problem	0.413	0.300	0.192	0.40	4.5 s
17	ros2_control framework	0.571	0.462	0.314	0.60	3.8 s
18	Franka Panda vs UR5e	0.450	0.279	0.213	1.00	15.3 s
19	Configuration space	0.735	0.679	0.459	0.80	3.9 s
20	Nav2 for mobile navigation	0.590	0.461	0.235	0.40	4.1 s

Table 6: Per-question evaluation results.

6.6 User Experience

The application provides a responsive, intuitive interface with several advanced features:

- **Chat-style interaction:** Users type questions naturally and receive formatted answers.
- **Example questions:** Randomized, clickable example questions help new users get started.
- **Source transparency:** Every answer includes an expandable "Source Documents" section.
- **Response time display:** Each answer shows generation latency.
- **Dark theme:** A modern, eye-friendly dark theme for extended use.

6.6.1 Multi-Turn Conversation Memory

The application maintains conversation context across turns by including the last 3 exchanges (up to 6 messages) in the LLM prompt. This enables natural follow-up questions such as asking "How does it compare to PRM?" after receiving an answer about RRT*—the model understands that "it" refers to RRT* from the conversation history. Long assistant responses are truncated to 500 characters in the history to stay within context limits.

6.6.2 Retrieval Confidence Indicator

Each answer displays a color-coded confidence indicator based on the best retrieval similarity score from ChromaDB. The raw relevance scores (0–1 range from L2 distance normalization) are scaled by 1.5× and mapped to three levels:

- **High (≥70%, green):** The knowledge base contains strongly relevant content.
- **Medium (45–69%, orange):** Partial coverage; the answer may be less detailed.
- **Low (<45%, red):** The topic is likely outside the knowledge base scope, with a warning displayed.

This transparency helps users gauge answer reliability and understand the system's limitations.

6.6.3 AI-Generated Follow-Up Suggestions

After each answer, GPT-4o-mini generates up to 3 follow-up questions that the user can click to continue exploring the topic. To ensure quality, each candidate follow-up undergoes two validation filters:

1. **Blocklist filter:** Rejects questions containing phrases that indicate out-of-scope topics (e.g., "compare to other", "traditional", "implement", "configure").
2. **Retrieval confidence check:** Each candidate is tested against the vector store; only questions with a top retrieval score $\geq$0.42 are shown.

This two-layer validation ensures that follow-up suggestions lead to questions the knowledge base can actually answer.

6.6.4 Answer Feedback System

Every assistant response includes thumbs-up and thumbs-down buttons, allowing users to provide feedback on answer quality. Feedback is stored in the session state and persists across the conversation. Once feedback is recorded, the buttons are replaced with a confirmation message. This feature demonstrates consideration for iterative quality improvement and user experience evaluation.

7. Deployment Strategy

7.1 Deployment Platform: Streamlit Cloud

The application is deployed on **Streamlit Cloud**, a free hosting platform purpose-built for Streamlit applications.

Deployed Application: https://roborag.streamlit.app/

7.2 Deployment Process

1. The complete project (including the pre-built ChromaDB vector store) is pushed to a public GitHub repository.
2. On Streamlit Cloud (https://share.streamlit.io/), the repository is connected and `app.py` is set as the main file.
3. The `OPENAI_API_KEY` is added as a secret in the Streamlit Cloud dashboard (Settings → Secrets).
4. Streamlit Cloud automatically installs dependencies from `requirements.txt` and launches the app.

7.3 Why Streamlit Cloud

7.4 Expected User Interaction

1. User opens the deployment link in any web browser.
2. The app loads with a welcome screen showing randomized example questions.
3. User types a robotics question or clicks an example.
4. Within 3–5 seconds, a grounded answer appears with: a retrieval confidence indicator, expandable source citations, response latency, and feedback buttons.

Feature	Benefit
Always on	No sleep/wake cycle — loads instantly when visited
Free tier	No cost for public apps
Persistent URL	Deployment link remains active indefinitely
Auto deploys	Git pushes trigger automatic redeployment
Secrets management	API keys stored securely and injected at runtime

Table 7: Streamlit Cloud advantages.

5. Below the answer, AI-generated follow-up suggestions let the user explore related topics with one click.
6. Multi-turn conversation memory enables natural follow-up questions that reference prior context.

8. Expected Outcomes and Challenges

8.1 Expected Outcomes

1. **Grounded, cited answers:** Technically accurate answers grounded in verified documentation with source citations.
2. **Demonstrated RAG effectiveness:** 146% improvement in ROUGE-1 and 154% improvement in Token F1 over the standalone LLM baseline.
3. **Responsive user experience:** Average response time of 4.89 seconds with multi-turn memory, confidence indicators, follow-up suggestions, and feedback buttons.
4. **Accessible deployment:** Streamlit Cloud (free) + OpenAI API (under $2 total).

8.2 Challenges Encountered and Solutions

Challenge	Impact	Solution
Initial API choice (Gemini free tier)	20 gen. requests/day, 100 embed requests/min made dev impractical	Switched to OpenAI GPT-4o-mini (~$2 total, generous rate limits)
Knowledge base coverage	Gaps lead to "I don't know" or poor retrieval	Authored 8 comprehensive docs (~15,000 words)
Chunk boundary issues	Split info causes incomplete retrieval	100-token overlap + semantic separators
Deployment memory	Streamlit Cloud 1 GB limit	Cloud-based embedding API; app uses only 341 MB
Evaluation rigor	Must fairly compare RAG vs. baseline	20 benchmark Q&As with hand-written references

Table 8: Challenges and solutions.

9. Resources Required

9.1 Software and Frameworks

Resource	Version	Purpose
Python	3.10+	Programming language
Streamlit	≥1.32.0	Web application framework
LangChain	≥0.3.0	RAG orchestration
langchain-openai	≥0.3.0	OpenAI API integration
langchain-chroma	≥0.2.0	ChromaDB integration
ChromaDB	≥0.5.0	Vector database
openai	≥1.30.0	OpenAI API client
rouge-score	≥0.1.2	ROUGE metrics
scikit-learn	≥1.4.0	ML utilities
psutil	≥5.9.0	Memory monitoring
python-dotenv	≥1.0.0	Environment variable management

Table 9: Software dependencies.

9.2 Models

Both models are accessed via API with no local download required:

- **OpenAI GPT-4o-mini** — text generation
- **OpenAI text-embedding-3-small** — vector embeddings

9.3 Data Sources (Curated Knowledge Base)

- Motion planning algorithms (RRT, RRT*, PRM, STOMP, CHOMP, TrajOpt)
- MoveIt 2 framework documentation
- ROS 2 fundamentals and architecture
- Robot manipulation and grasping techniques
- Robot kinematics and dynamics
- Path planning fundamentals
- SLAM and perception for robotics
- Common robot platforms (Franka Panda, UR5e, KUKA iiwa, etc.)

9.4 Hardware and Cost

- **Development:** Any machine with Python 3.10+ and internet access
- **Deployment:** Streamlit Cloud free tier (1 GB RAM, shared CPU) — no GPU required
- **API cost:** Under $2 total for the entire project lifecycle

10. Conclusion

10.1 Key Takeaways

RoboRAG demonstrates that Retrieval-Augmented Generation is an effective approach for building domain-specific question-answering systems in technical fields. By grounding LLM responses in curated, verified documentation, the system provides accurate, cited answers that users can trust and verify—addressing the fundamental hallucination problem that limits the usefulness of standalone LLMs in specialized domains.

The project achieved its core objectives:

1. **Functional web application:** A responsive Streamlit chat interface that answers robotics questions with cited sources, deployed on Streamlit Cloud with a persistent URL. Enhanced with multi-turn conversation memory, retrieval confidence indicators, AI-generated follow-up suggestions, and user feedback buttons.
2. **Quantified RAG advantage:** The full 20-question evaluation demonstrates a **146% improvement in ROUGE-1** and **154% improvement in Token F1** when using RAG versus the standalone LLM.
3. **Strong retrieval quality:** Precision@5 of 77.0% confirms that the embedding model and chunking strategy effectively identify relevant passages.
4. **Efficient and accessible:** Average latency of 4.89 s, memory usage of 341 MB, and total API cost under $2.

10.2 Potential Improvements and Future Enhancements

1. **Expanded knowledge base:** Incorporate live scraping of MoveIt 2 and ROS 2 docs, more arXiv papers, and community Q&A threads.
2. **Hybrid retrieval:** Combine dense (embedding-based) retrieval with sparse (BM25) retrieval for better handling of specific technical terms.
3. **Persistent feedback storage:** Extend the current session-based feedback system to a persistent database for long-term quality tracking.
4. **Fine-tuned embeddings:** Train an embedding model on robotics text for improved domain-specific retrieval.
5. **Upgraded LLM:** Test with GPT-4o or newer models for improved generation quality on complex questions.
6. **User-uploaded documents:** Allow users to upload their own robotics documents to expand the knowledge base per session.

References

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *NeurIPS*, 2020.

[2] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.

[3] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Iowa State University, Tech. Rep. TR 98-11, 1998.

[4] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Intl. J. Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

[5] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Cambridge Univ. Press, 2017.

[6] LangChain, https://github.com/langchain-ai/langchain, 2023.

[7] Chroma, https://github.com/chroma-core/chroma, 2023.

[8] OpenAI API Documentation, https://platform.openai.com/docs, 2024.

[9] Streamlit Documentation, https://docs.streamlit.io/, 2024.